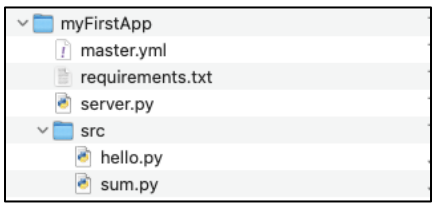
ENGR -222: Intelligent Systems II **Week 4 Lab**

The goals of this lab are

1. To set up a simple directory for our first application
2. To create our first OpenAPI specification file using the Swagger Editor tool.
3. To get introduced to JetStream2, the cloud-based infrastructure will use to deploy our service.

Part I. Directory Set-Up

Keeping organized is crucial. Let's setup a workspace for our first application deployment. This should be included in the Week 4 Github folder. For now, you should create a ***myFirstApp*** folder with an ***src*** folder inside. At the end of the day, the following items will be in your workspace:



Part II. OpenAPI Specification

The Swagger Editor is a tool for us to create our OpenAPI Specification file. This file defines and documents our application interface while complying with REST infrastructure. The Specification file will be exported as a YAML file.

This file defines endpoints and links them to functions within our service that get accessed when the path/endpoint is requested.

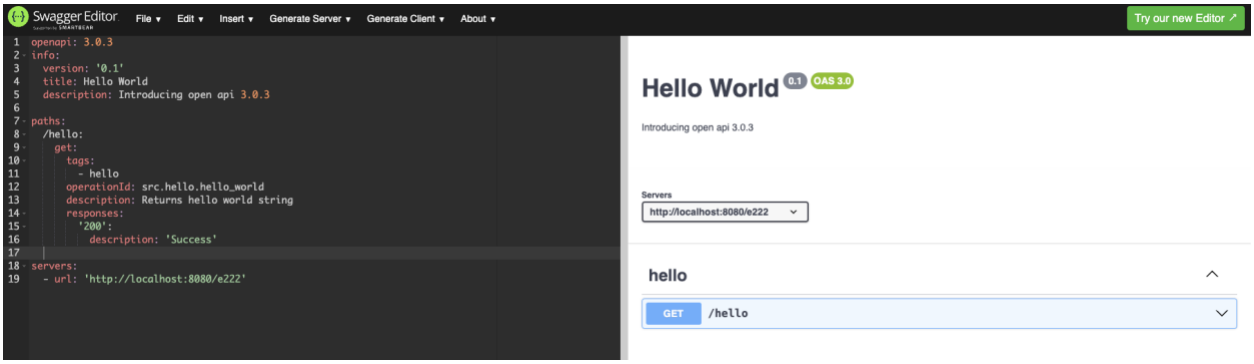
```yaml
openapi: 3.0.0
info:
  version: '0.0'
  title: Hello World
  description: Introducing open api 3.0.0

paths:
  /hello:
    get:
      tags:
        - hello
      operationId: src.hello.hello_world
      description: Returns hello world string
      responses:
        '200':
          description: 'Success'
  /sum/<arg1>/<arg2>:
    get:
      tags:
        - SUM
      operationId: src.sum.sum
      description: Returns the sum of two numbers
      responses:
        '200':
          description: 'Success'
```

Endpoints/Resources

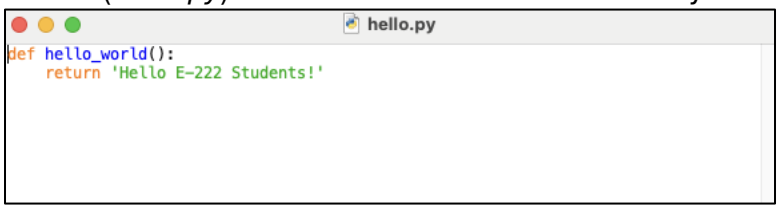
Directory.Filename.Function

Sending arguments

1. Navigate to the Swagger Editor and let's create this file for our service. Try it out with the editor interface. When you are finished, saved the file in your workspace (***myFirstApp***).
2.

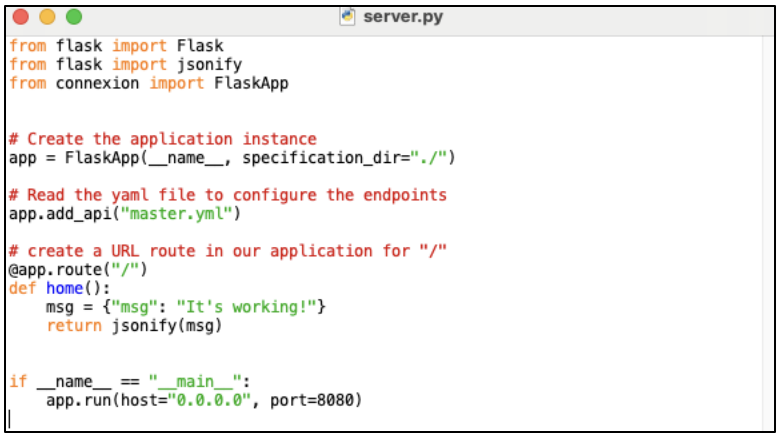


3. Our interface was missing the functions that provide the request execution. We need to create those (*hello.py*) and save them in our ***src*** directory.



4. The last thing we need is to create an app and deploy it. We will be using Flask, which is a web application framework. A web framework is a software framework that is designed to support the development of web applications including web services, web resources, and web APIs. Let's code up our ***server.py*** file.

https://flask.palletsprojects.com/en/stable/



<u>Once you have completed the demo. Add, commit, push these files to your individual Github repository.</u>

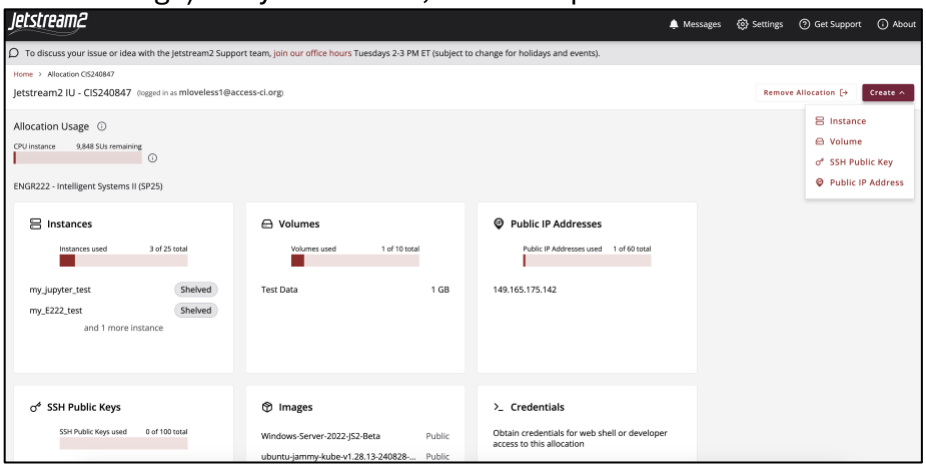
Part III. JetStream2

It seems that we have all the pieces to deploy the service. Could we deploy it on this local machine? Yes, but it is not a long-term solution. Let's use JetSteam2, a cloud-based infrastructure. *"Cloud-based and on-demand, the Jetstream2 environment also includes discipline-specific apps. You can even create virtual machines that look and feel like your lab workstation or home machine—but with thousands of times the computing power."* https://jetstream-cloud.org/

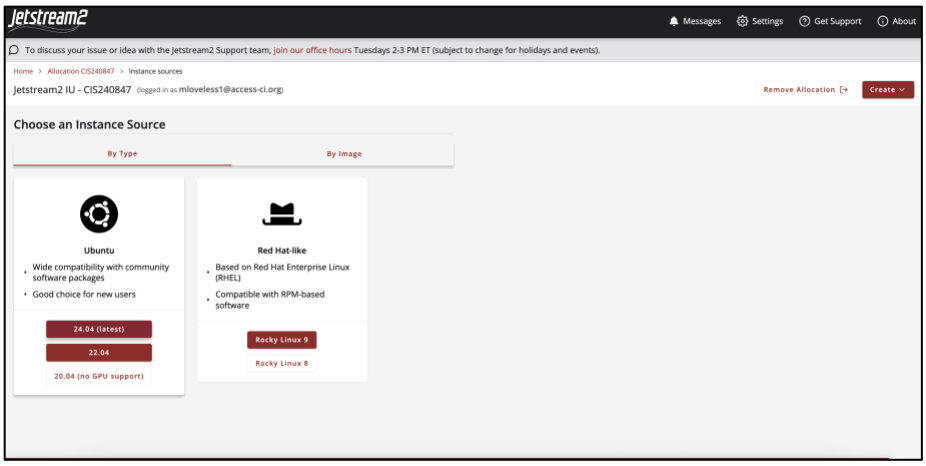
Fun Facts about JetStream2:

- Jetstream2 is led by staff of the Pervasive Technology Institute at Indiana University, under the leadership of Jetstream2 Principal Investigator and Director of Advanced Cyberinfrastructure, David Y. Hancock.
- Beth Plale (ISE Dept Chair) is executive director of the Pervasive Technology Institute at Indiana University. PTI is Indiana University's flagship organization for research and development in cyberinfrastructure, informatics, and computer science in support of science and engineering research, artistic creativity, and cybersecurity.

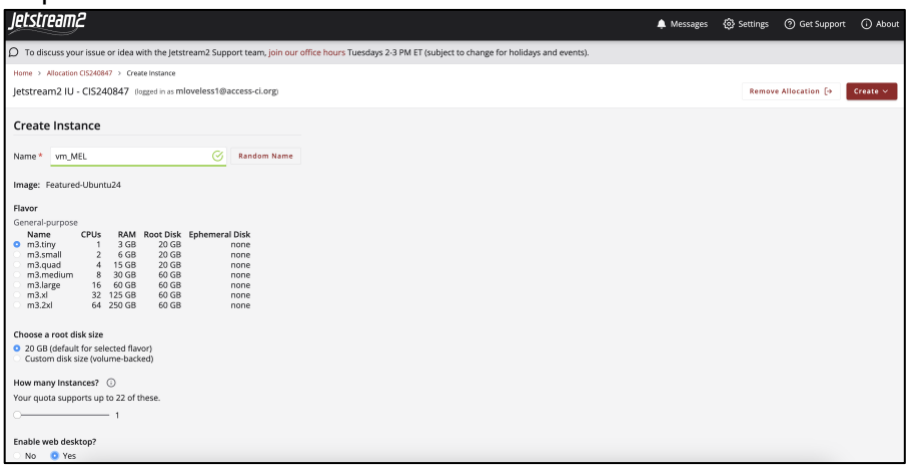
JetStream2 allows us to create virtual environments to test and deploy our service. The dashboard below allows you to create instances of virtual machines as well as volumes (memory for data storage). As you can see, there is a public IP address for each instance.



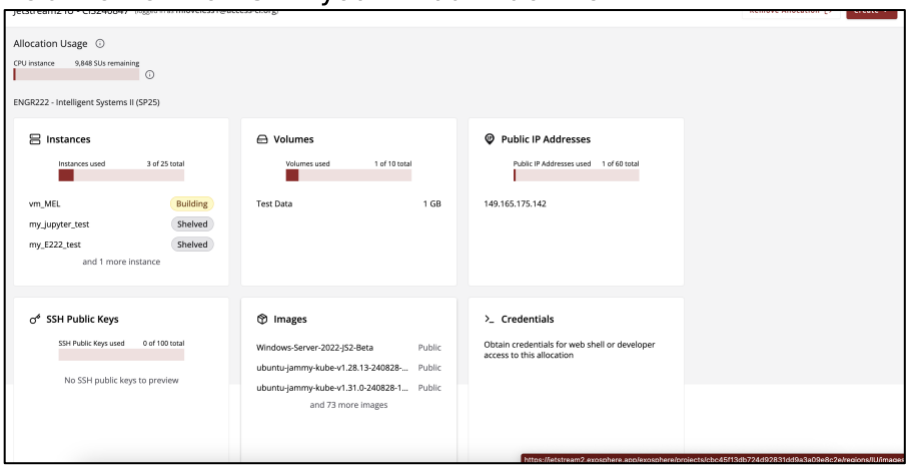
When you create an instance, you can choose an OS (or an Image that you might have created in the past). We will choose the latest Ubuntu version as the source.



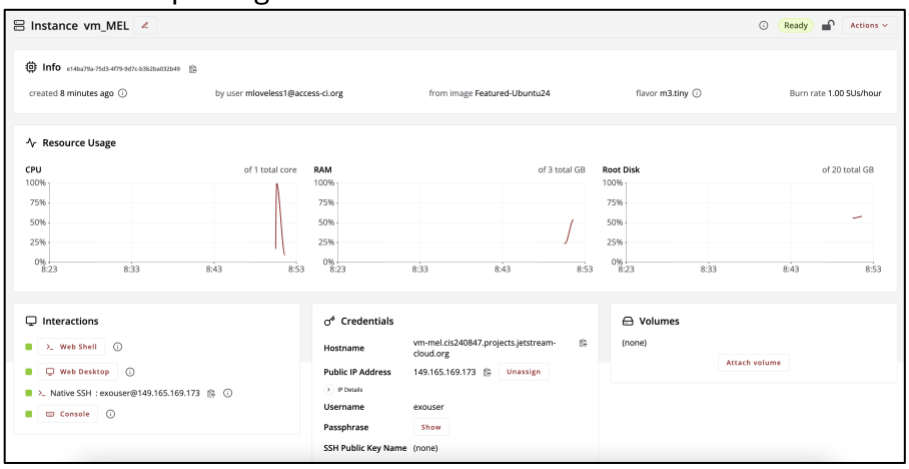
The next screen will ask you to choose the specifications for your virtual machine. Enter your name (vm_LASTNAME), choose m3.tiny, 20 GB Root Disk Size, 1 instance and YES for the web desktop.



It will then take a moment to BUILD your virtual machine.

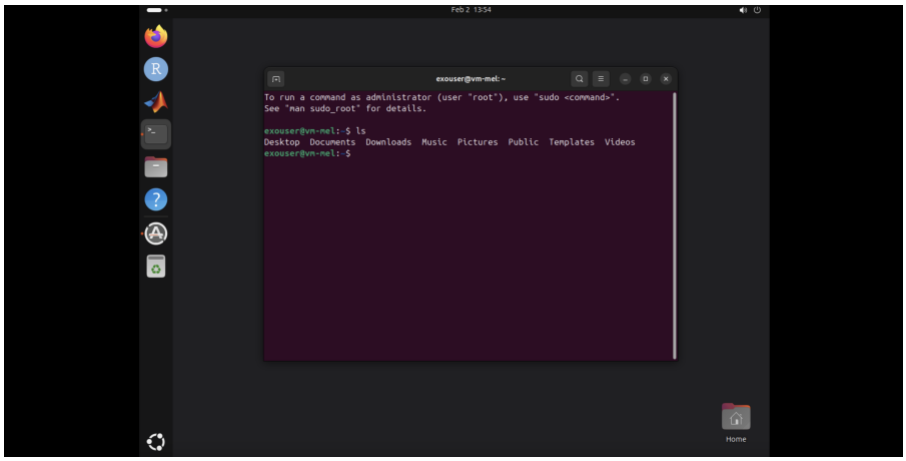


Once built, your instance dashboard is shown similar to below. Take a moment and look at all the information it is reporting.



Let's tour your instance. Select Web Desktop under Interactions. It should bring your to a Desktop interface. It should look and feel like your own personal computer. What do you notice?

You can even pull up the terminal within the the Desktop Interface. Pull it up and execute some basic Linux commands.



Let's exit the Desktop Interaction and pull up the Shell from the Interaction section of the Instance Dashboard page. It should look JUST like your terminal in your Desktop interface.

Navigate to Documents. Create a directory called work.

```
cd Documents/
mkdir work
cd work/
```

Let's connect your Github repositories here as well.

```
git clone link-to-E222-student-repo
git clone link-to-your-student-repo
```

Navigate into your personal repo to your ***myFirstApp*** folder. What happens when you try to run server.py?

```
python3 server.py
```

You probably received some dependency errors – packages were not installed. Remember this is a fresh machine! BUT before we start installing the required packages, lets create a virtual environment to "enclose" our python project. What is a python virtual environment? It is used to contain a specific Python interpreter and software libraries and binaries which are needed to support a project (library or application). These are by default isolated from software in other virtual environments and Python interpreters and libraries installed in the operating system. They are contained in a directory, conventionally named `.venv` or `venv` in the project directory. The nice thing is that they are considered disposable – it should be simple to delete and recreate it from scratch. Let's use one here.

IMPORTANT SIDE BAR! Add to gitignore file: `.venv/`

```
vim .gitignore
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

You should have a different prefix in your command line now (.venv). Ok, now let's do the pip installs needed for this project.

```
pip install Flask
pip install connexion[flask, swagger-ui, uvicorn]
```

What happens if you need to recreate your VM instance? You lose ALL of these installs! Let's save them somewhere so we know what we had to install to make this app work.

```
pip freeze > requirements.txt
```

Later, we can run

```
pip install -r requirements.txt
```

Cool, eh?

Now, this might be too much...so you can always just include what you installed above in your requirements file.

Student Task

At this point we should know how to define an API endpoint (e.g., /paths). We should also know how the operationID ties the python function to the endpoint.

Your task is to create a python function that will return a string that details your model information. Edit your API specification file to include an endpoint for your application.

Take a look here at the sum function, and keep in mind for us to use python within our API we have to make functions. As we think about our project we need to think about what functions we will need to create.

```
def model_info():
    details = "The model used .... include hyper parameters,
libraries, etc..."
    return str(details)
```

Make sure you put in another GET request with the appropriate path and operationID.